

Introduction to Computer Vision (83551) - Assignment 2A

In this assignment you will build and train several core deep learning models, starting from low-level NumPy code and moving toward PyTorch. You will see how design choices affect training stability, overfitting, and performance on real image data.

Question 1: Batch Normalization

For this task, please follow the steps outlined in the file `BatchNormalization.ipynb`. This task includes filling in the code inside:

`BatchNormalization.ipynb` (inline questions)

`icv83551/layers.py`

`icv83551/classifiers/fc_net.py`

Note: with the free resources plan the code should run in about 3 minutes.

Question 2: Dropout

For this task, please follow the steps outlined in the file `Dropout.ipynb`. This task includes filling in the code inside:

`icv83551/layers.py`

`icv83551/classifiers/fc_net.py`

and also, code and inline questions inside `Dropout.ipynb`

Note: In `Dropout.pynb` the total running time should be around 1 minute.

Question 3: Convolutional Neural Networks

For this task, please follow the steps outlined in the file `ConvolutionalNetworks.ipynb`. This task includes filling in the code inside:

`icv83551/layers.py`

`icv83551/classifiers/cnn.py`

and also, code and inline questions inside ConvolutionalNetworks.ipynb

Note: Total running time is about 60 minutes.

Question 4: PyTorch on CIFAR-10

For this task, please follow the steps outlined in the file PyTorch.ipynb.

Fill in the code inside the file: PyTorch.ipynb.

Note: Total running time is less than 3 minutes and should be mainly on extracting features.

Question 5: Image Captioning with Vanilla RNNs

For this task, please follow the steps outlined in the file RNN_Captioning_pytorch.ipynb. In addition, fill the code inside the file:

icv83551/rnn_layers_pytorch.py

icv83551/classifiers/rnn_pytorch.py

Note: Total running time should be about 1 minute.

Important Note:

The following IPython magic commands are intended for local Jupyter environments to auto-reload changed modules:

```
%load_ext autoreload
```

```
%autoreload 2
```

In Google Colab this autoreload extension may not be available or compatible, which can cause errors when running the notebook. **Please delete or comment out** the two lines above so the code will run properly in Colab.

Required Submission Files:

Prepare the following files:

- a2_code_submission.zip: Generate by running **collect_submission.ipynb** inside the assignment2 directory.
- Simpler .txt file named [Your_Name_ID].txt with a link to your Github project. Make sure your link is publicly available.

Submit the following files (generated by collect_submission.ipynb):

1. To the VPL (Assignment 2 - Automated testing):

1. BatchNormalization.ipynb
2. Dropout.ipynb
3. ConvolutionalNetworks.ipynb
4. PyTorch.ipynb
5. RNN_Captioning_pytorch.ipynb

2. To Assignment 2 – submit only the txt file with your Github link.

In addition, you should extract all the files inside a2_code_submission.zip and upload them into your Github project. A link into your Github project should be in collect_submission.ipynb and also in a simpler .txt file named [Your_Name_ID].txt.

The submission must include written answers (inside ipynb files), all required uploaded files, and the corresponding code, so that every component of the assignment can be reviewed and fully graded.

VPL Submission System

Submit your assignment through the VPL module in Moodle (Lambda), our dedicated platform for programming tasks.

Academic Integrity Checks

VPL includes a built-in plagiarism detection tool that compares submissions within the course—and optionally across prior semesters—to identify similarities and ensure originality. Instructors review flagged cases manually for fair assessment, upholding academic honesty standards.